Slides and example codes are available:
https://github.com/mint-lab/cv_tutorial

Image Segmentation

Sunglok Choi, Assistant Professor, Ph.D.
Computer Science and Engineering Department, SEOULTECH
sunglok@seoultech.ac.kr | https://mint-lab.github.io/

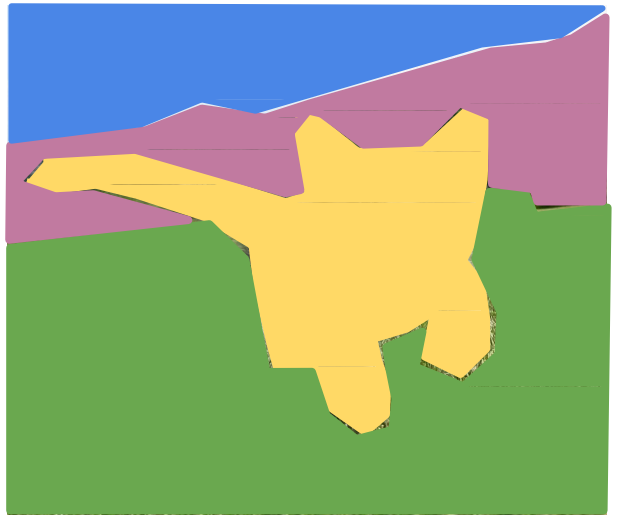
Computer Vision Tasks

Image Classification

CAT

No spatial extent

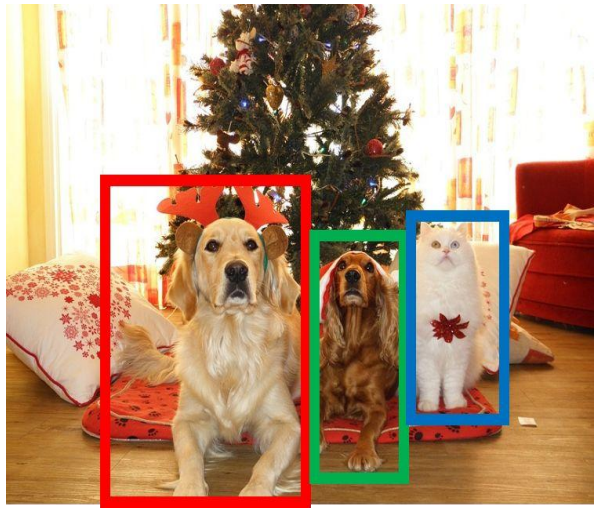
Semantic Segmentation



GRASS, CAT, TREE, SKY

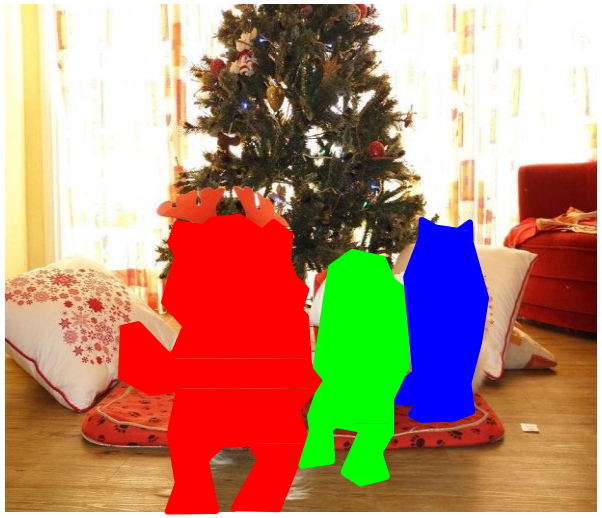
No object, just pixels

Object Detection



DOG, DOG, CAT

Instance Segmentation



DOG, DOG, CAT

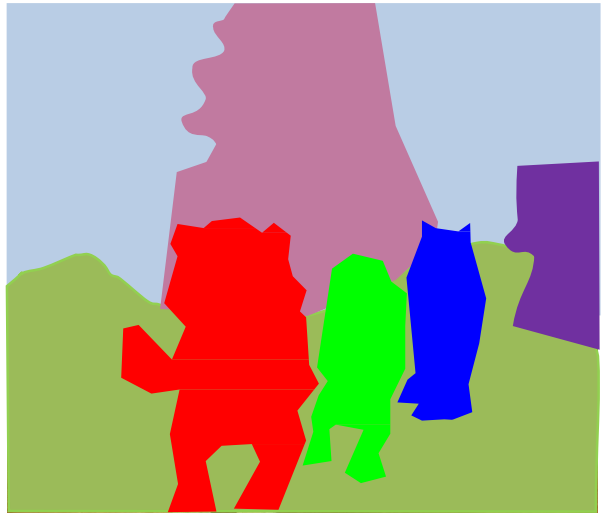
Multiple objects

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

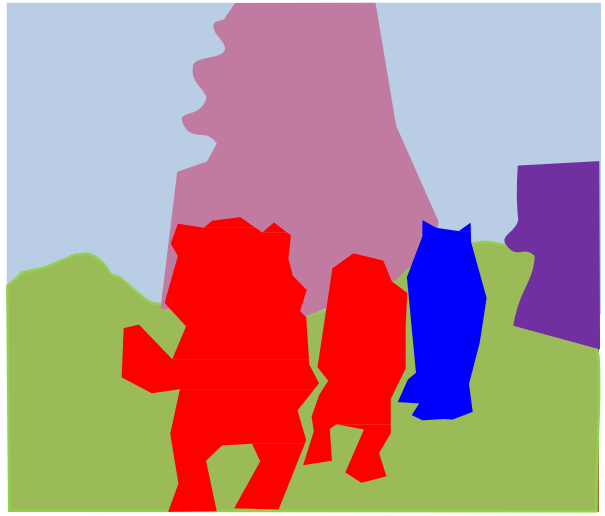
Image Segmentation

- **Image segmentation** is a task to partition image into multiple image segments (a.k.a. regions or objects) as sets of pixels.

Panoptic Segmentation = Semantic Segmentation + Instance Segmentation



DOG, DOG, CAT
FLOOR, SOFA, TREE, WALL



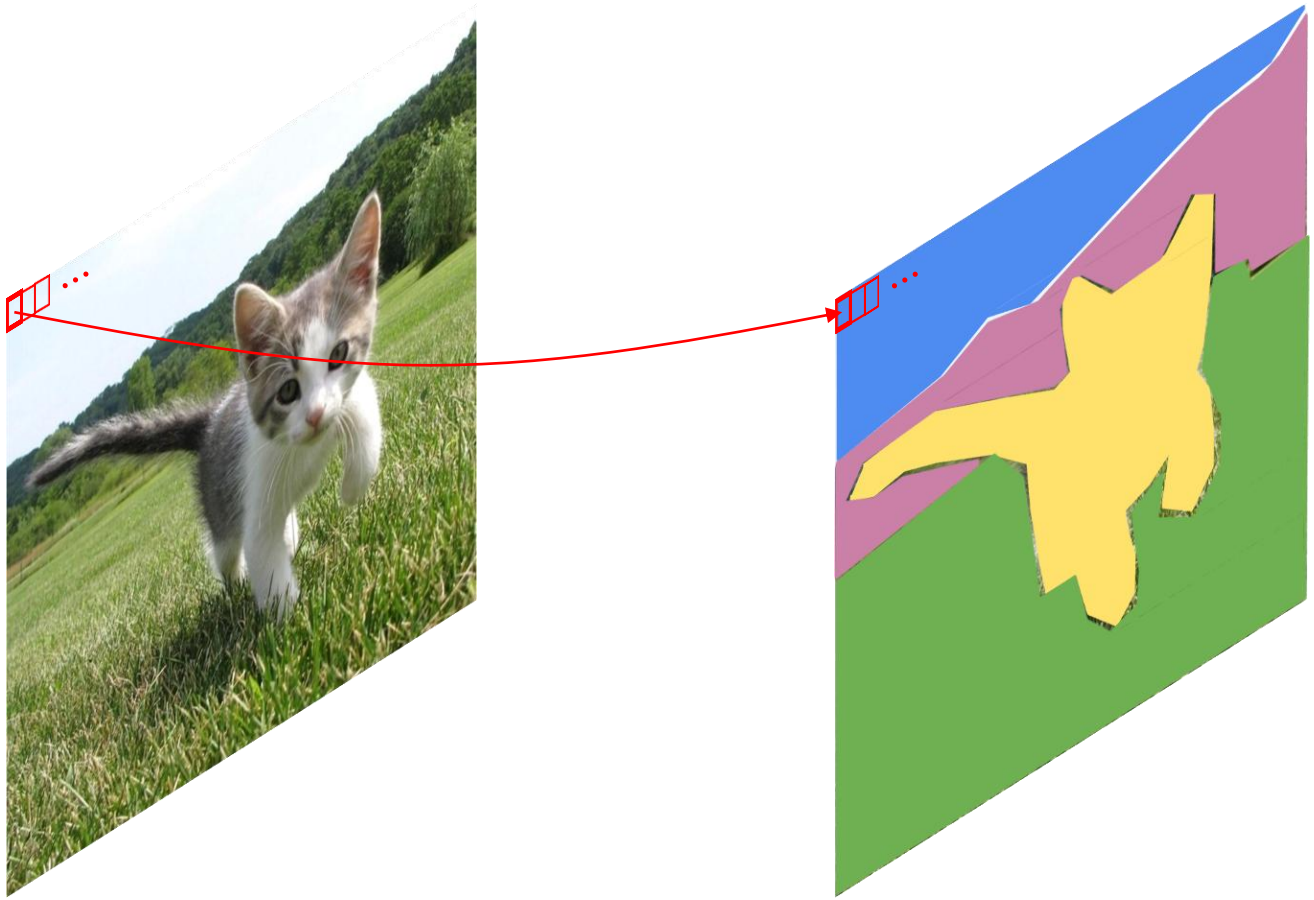
DOG, DOG, CAT
FLOOR, SOFA, TREE, WALL

DOG, DOG, CAT

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

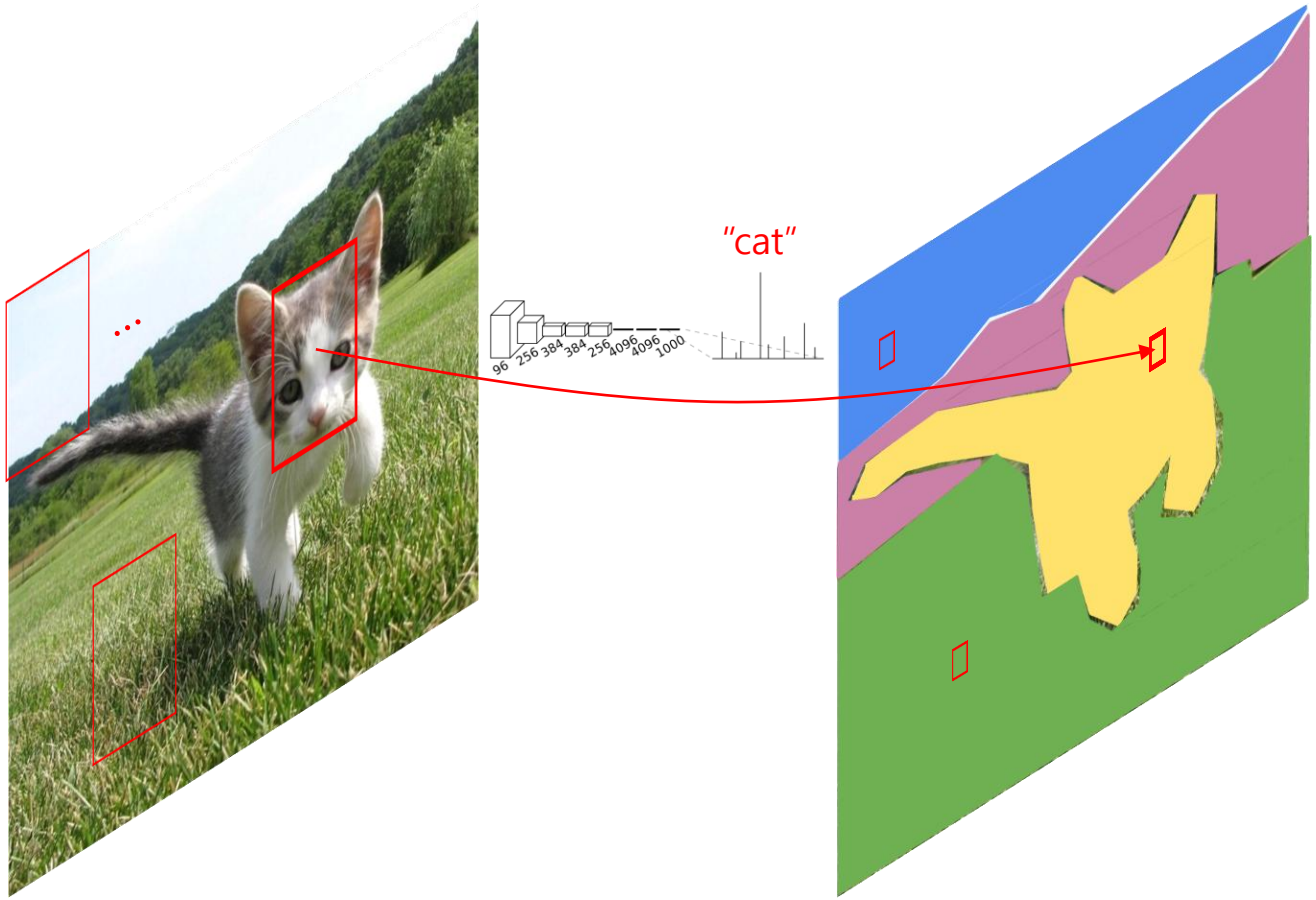
Semantic Segmentation

- Idea: Focusing one pixel
- Problem: This approach is a kind of pixel-wise thresholding. No way to recognize context!



Semantic Segmentation

- Idea: Using an image classifier for each image regions with sliding window
- Problem: Too long inference time (W x H x classification time)



FCN (Fully Convolutional Networks; 2015)

- Key idea: Fully convolutional networks with *transposed convolution* (a.k.a. deconvolution)
  - Note) It does not use any fully connected (shortly FC) layer.
  - A CNN backbone network is applied to the full image only once.

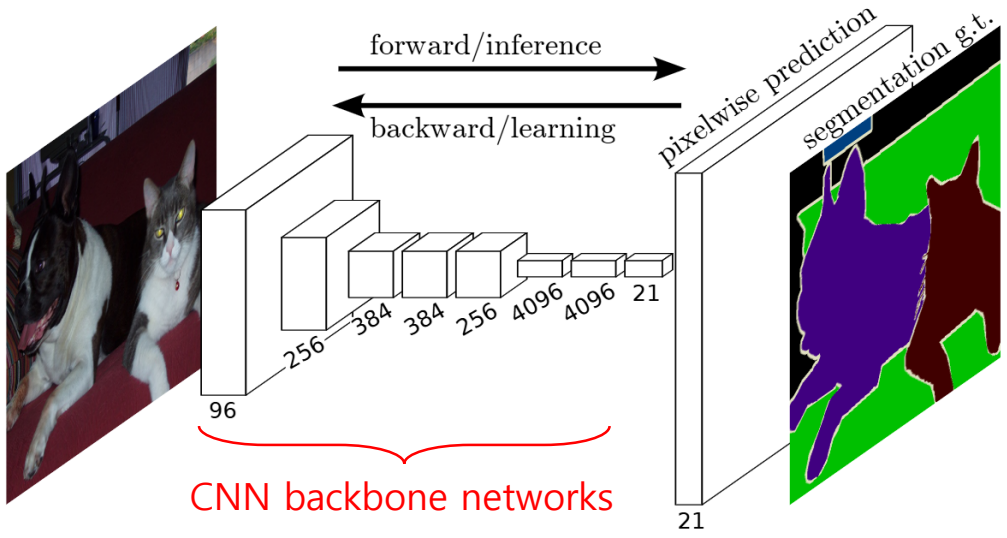


Figure 1. Fully convolutional networks can efficiently learn to make dense predictions for per-pixel tasks like semantic segmentation.

Table 1. We adapt and extend three classification convnets to segmentation. We compare performance by mean intersection over union on the validation set of PASCAL VOC 2011 and by inference time (averaged over 20 trials for a 500 × 500 input on an NVIDIA Tesla K40c). We detail the architecture of the adapted nets as regards dense prediction: number of parameter layers, receptive field size of output units, and the coarsest stride within the net. (These numbers give the best performance obtained at a fixed learning rate, not best performance possible.)

	FCN-AlexNet	FCN-VGG16	FCN-GoogLeNet[4]
mean IU	39.8	**56.0**	42.5
forward time	50 ms	210 ms	59 ms
conv. layers	8	16	22
parameters	57M	134M	6M
rf size	355	404	907
max stride	32	32	32

Image: Long et al., **Fully Convolutional Networks for Semantic Segmentation**, CVPR, 2015 arXiv / T-PAMI, 2017 DOI

FCN (Fully Convolutional Networks; 2015)

- Key idea: Fully convolutional networks with *transposed convolution* (a.k.a. deconvolution)
  - Trainable upsampling with ***transposed convolution*** (a.k.a. deconvolution)

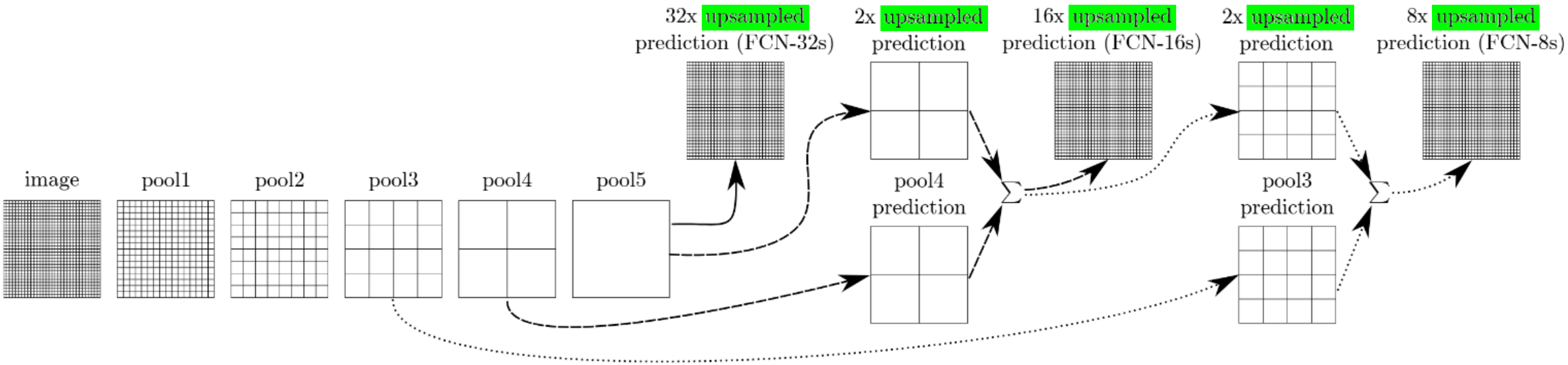


Figure 3. Our DAG nets learn to combine coarse, high layer information with fine, low layer information. Layers are shown as grids that reveal relative spatial coarseness. Only pooling and prediction layers are shown; intermediate convolution layers (including our converted fully connected layers) are omitted. Solid line (FCN-32s): Our single-stream net, described in Section 4.1, upsamples stride 32 predictions back to pixels in a single step. Dashed line (FCN-16s): Combining predictions from both the final layer and the `pool4` layer, at stride 16, lets our net predict finer details, while retaining high-level semantic information. Dotted line (FCN-8s): Additional predictions from `pool3`, at stride 8, provide further precision.

Image: Long et al., **Fully Convolutional Networks for Semantic Segmentation**, CVPR, 2015 arXiv / T-PAMI, 2017 DOI

FCN (Fully Convolutional Networks; 2015)

- Key idea: Fully convolutional networks with ***transposed convolution*** (a.k.a. deconvolution)
  - Trainable upsampling with ***transposed convolution*** (a.k.a. deconvolution)

Table 2. Comparison of skip FCNs on a subset of PASCAL VOC2011 validation[7]. Learning is end-to-end, except for FCN-32s-fixed, where only the last layer is fine-tuned. Note that FCN-32s is FCN-VGG16, renamed to highlight stride.

	pixel acc.	mean acc.	mean IU	f.w. IU
FCN-32s-fixed	83.0	59.7	45.4	72.0
FCN-32s	89.1	73.3	59.4	81.4
FCN-16s	90.0	75.7	62.4	83.0
FCN-8s	**90.3**	**75.9**	**62.7**	**83.2**

Table 3. Our fully convolutional net gives a 20% relative improvement over the state-of-the-art on the PASCAL VOC 2011 and 2012 test sets, and reduces inference time.

	mean IU VOC2011 test	mean IU VOC2012 test	inference time
R-CNN [12]	47.9	-	-
SDS [16]	52.6	51.6	~ 50 s
FCN-8s	**62.7**	**62.2**	~ **175 ms**

Note) Intersection over Union (IoU, IU)

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$

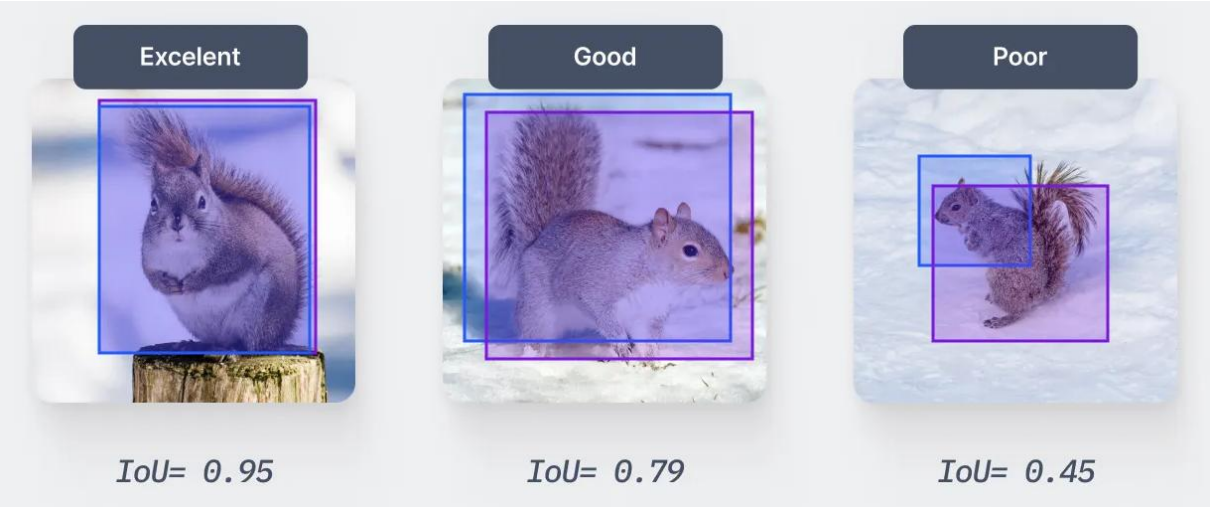
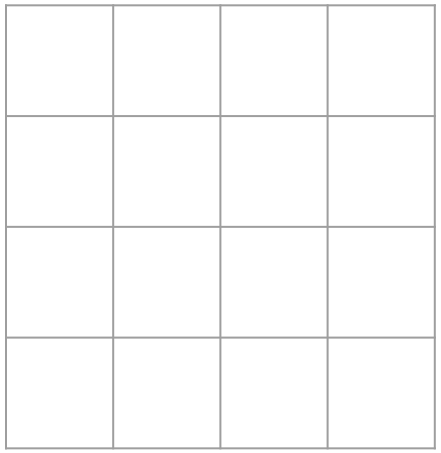


Image: V7 Blog

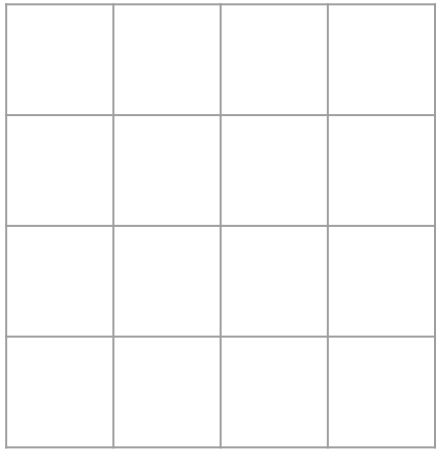
Image: Long et al., **Fully Convolutional Networks for Semantic Segmentation**, CVPR, 2015 arXiv / T-PAMI, 2017 DOI

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



Input: 4 x 4

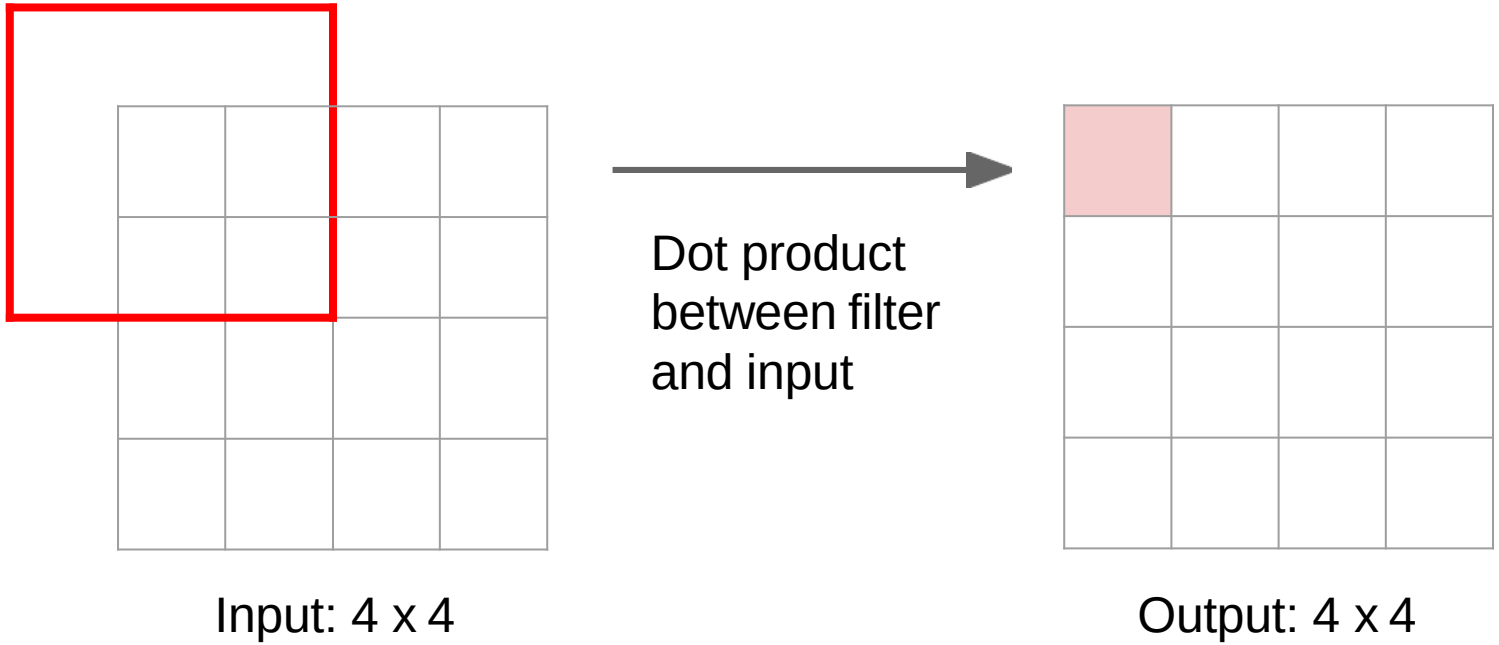


Output: 4 x 4

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling

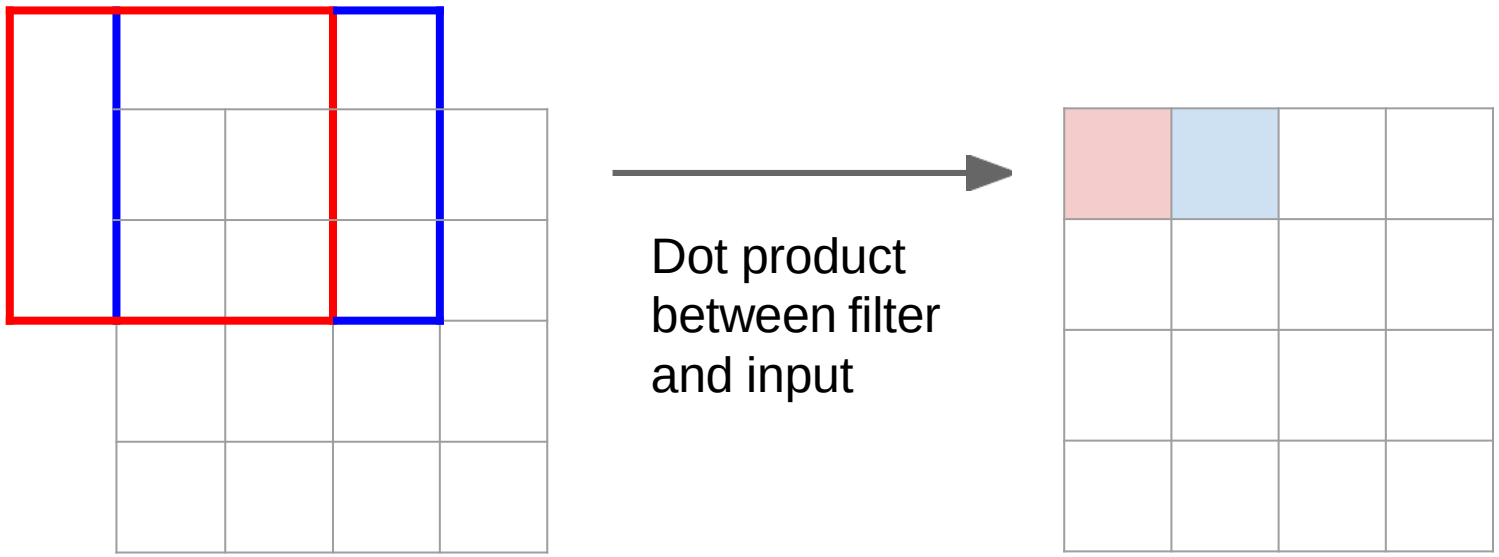
Recall: Normal 3 x 3 convolution, stride 1 pad 1



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 1 pad 1



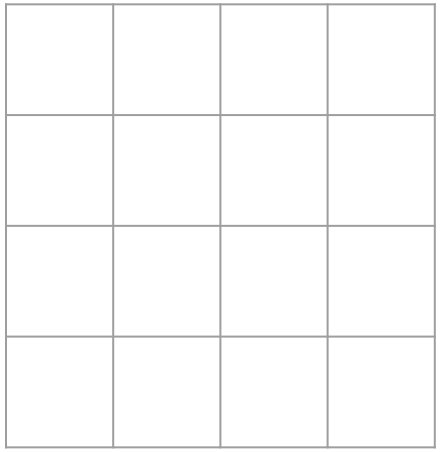
Input: 4 x 4

Output: 4 x 4

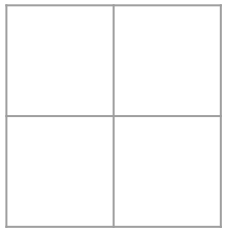
Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Input: 4 x 4

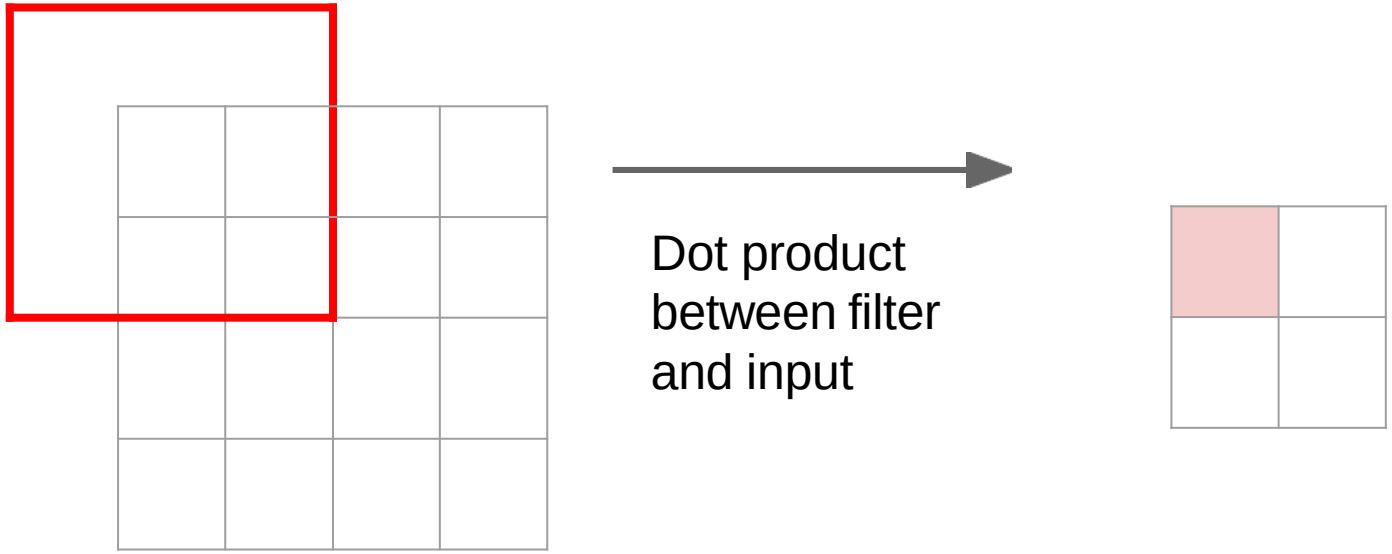


Output: 2 x 2

Slide: Stanford CS231n: Deep Learning for Computer Vision

Learnable Upsampling

Recall: Normal 3 x 3 convolution, stride 2 pad 1



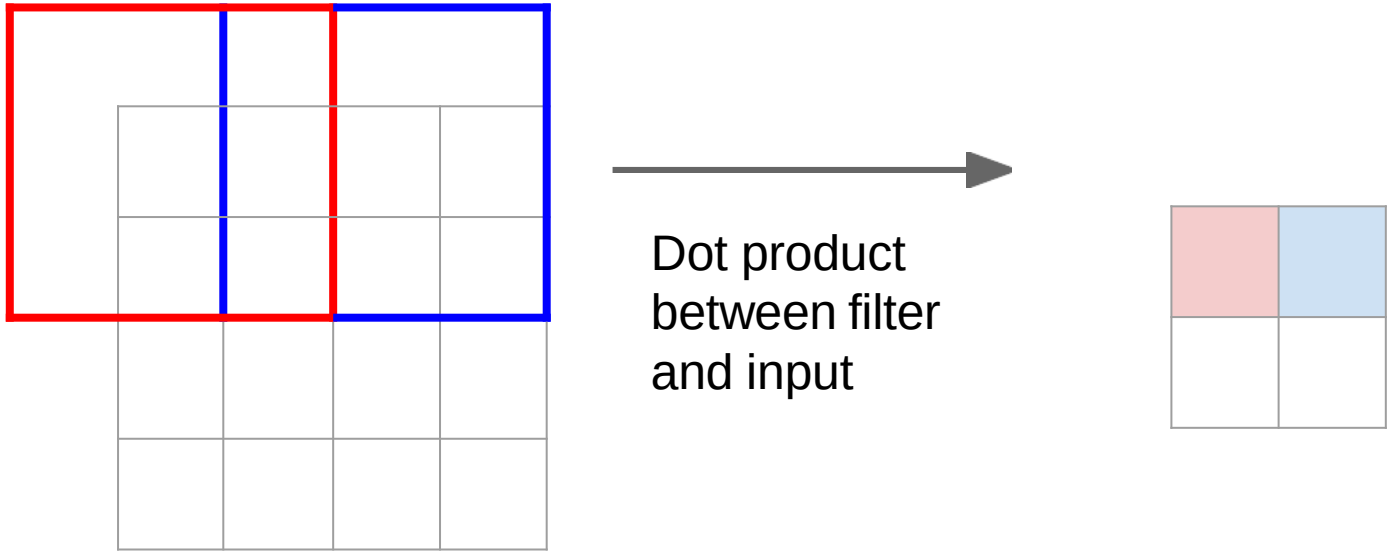
Input: 4 x 4

Output: 2 x 2

Slide: Stanford CS231n: Deep Learning for Computer Vision

Learnable Upsampling

Recall: Normal 3 x 3 convolution, <u>stride 2</u> pad 1



Input: 4 x 4

Output: 2 x 2

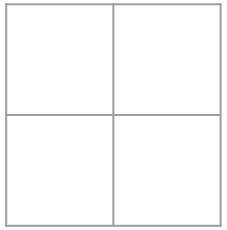
Stride gives ratio between movement in input and output

We can interpret strided convolution as “learnable downsampling”.

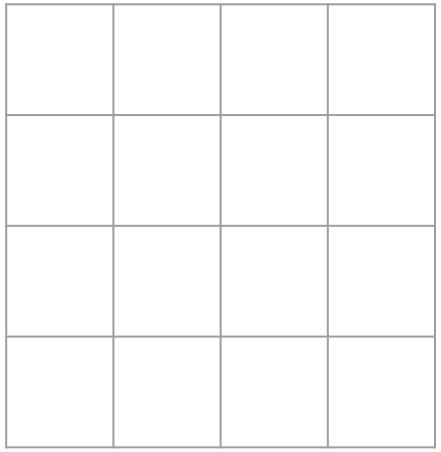
Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Input: 2 x 2

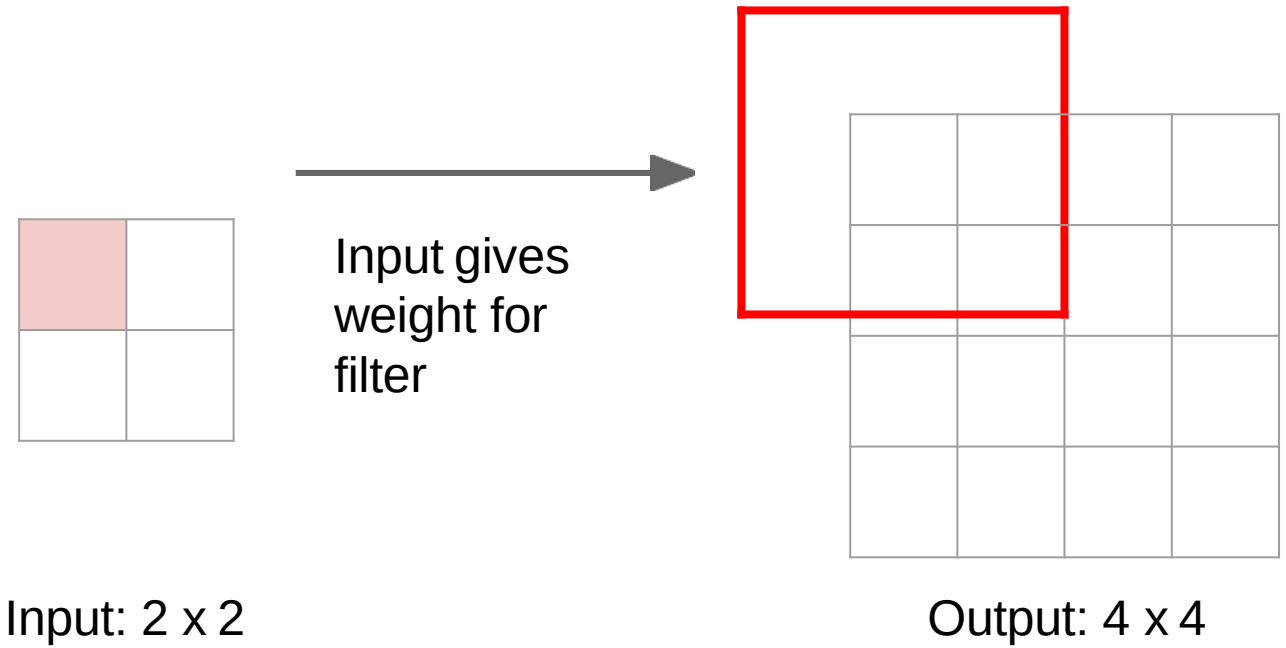


Output: 4 x 4

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling: Transposed Convolution

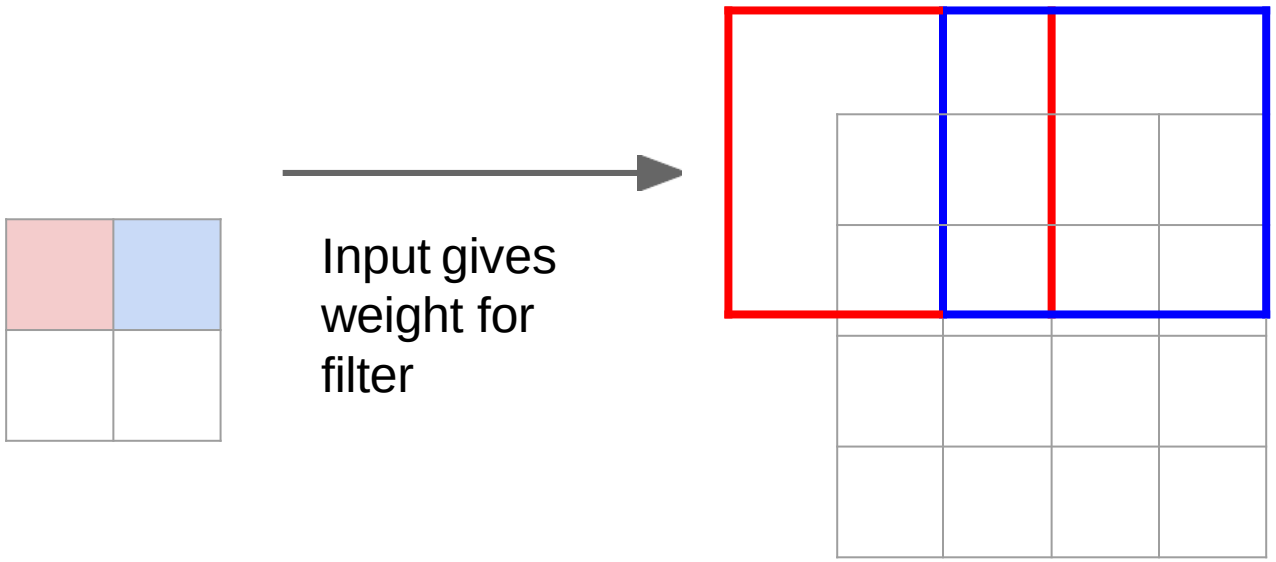
3 x 3 **transposed** convolution, stride 2 pad 1



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Input: 2 x 2

Output: 4 x 4

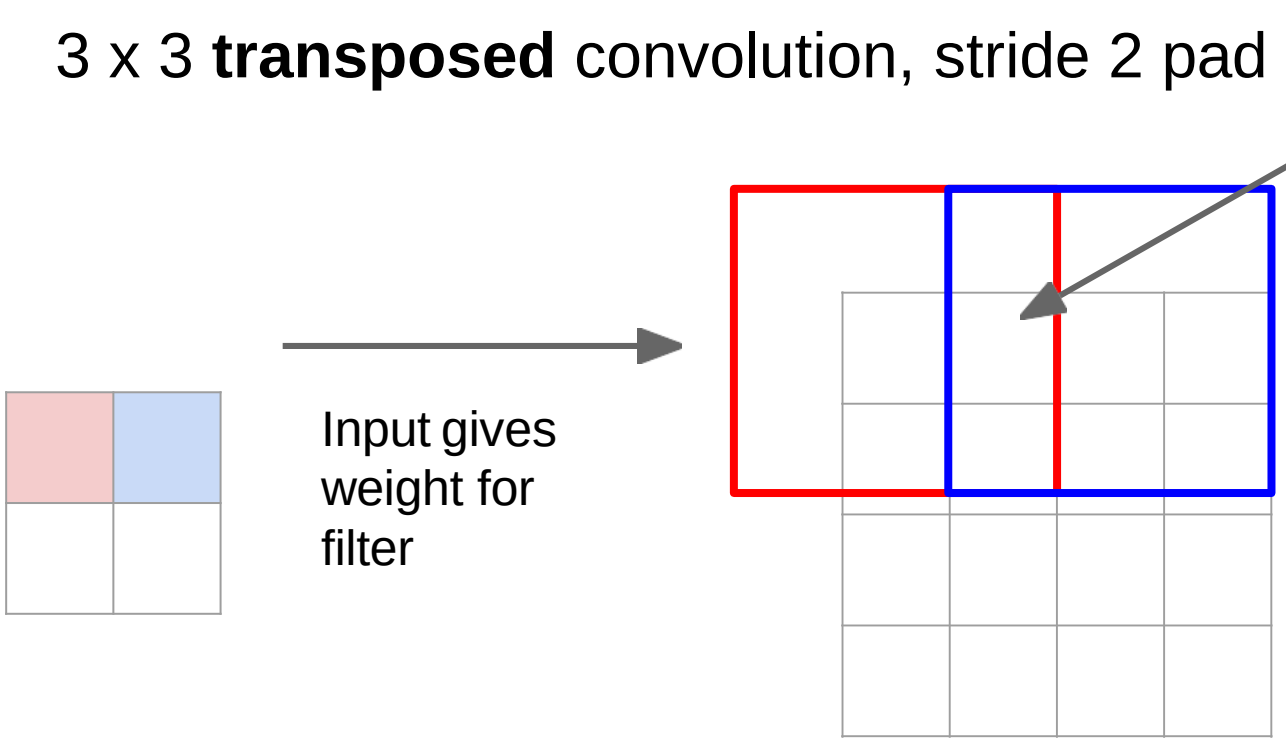
Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

Stride gives ratio between movement in output and input

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling: Transposed Convolution

3 x 3 **transposed** convolution, stride 2 pad 1



Sum where output overlaps

Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

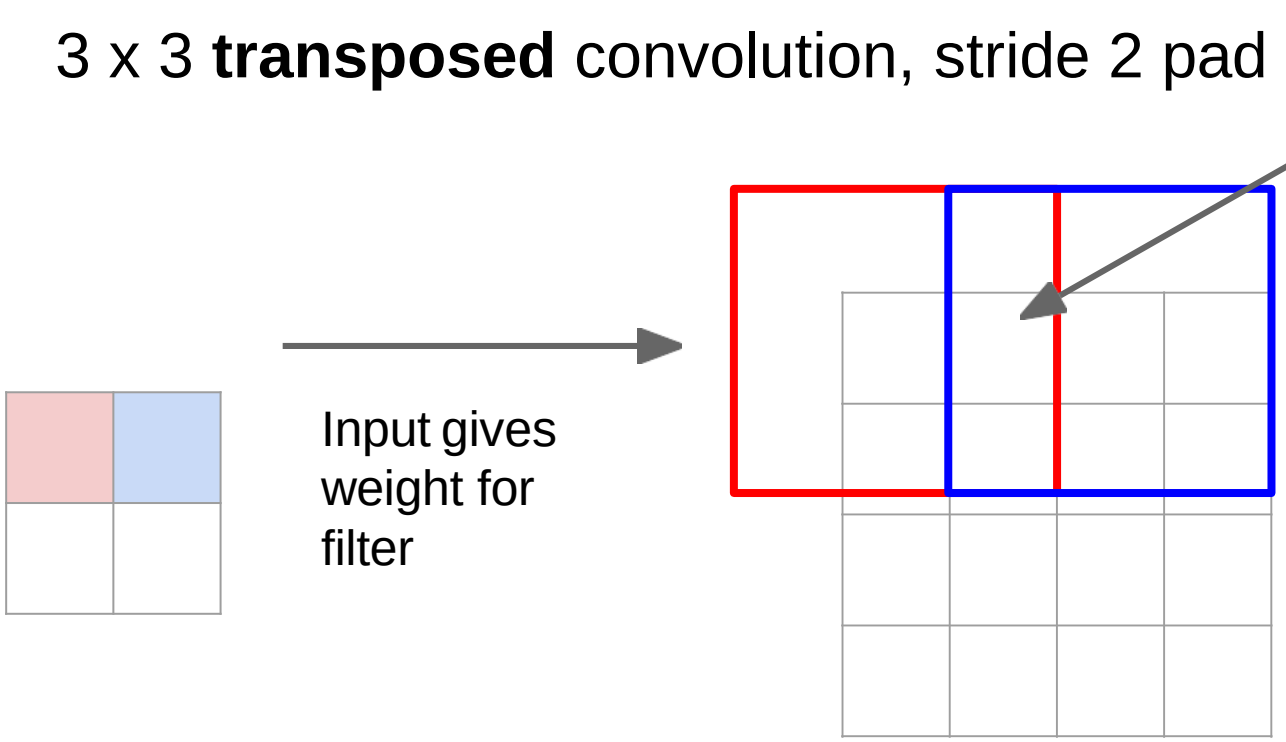
Stride gives ratio between movement in output and input

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

Learnable Upsampling: Transposed Convolution

Q: Why is it called transposed convolution?

3 x 3 **transposed** convolution, stride 2 pad 1



Sum where output overlaps

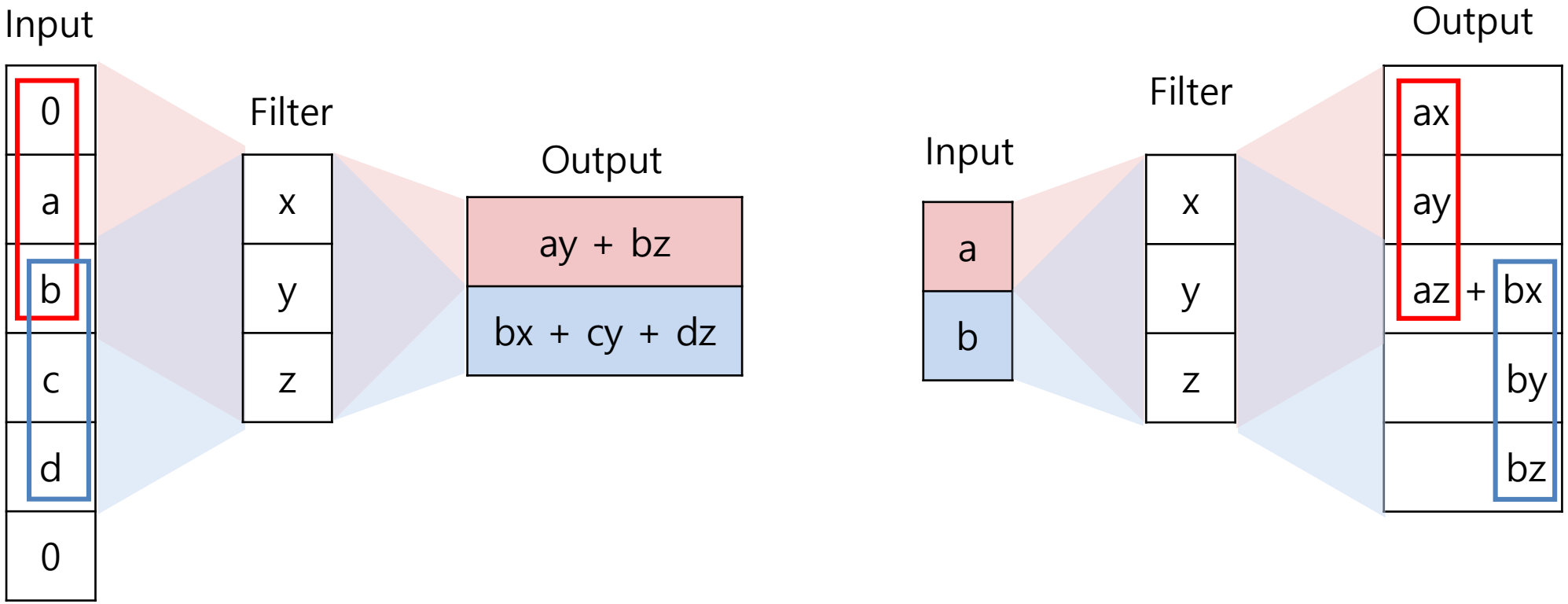
Filter moves 2 pixels in the <u>output</u> for every one pixel in the <u>input</u>

Stride gives ratio between movement in output and input

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

FCN (Fully Convolutional Networks; 2015)

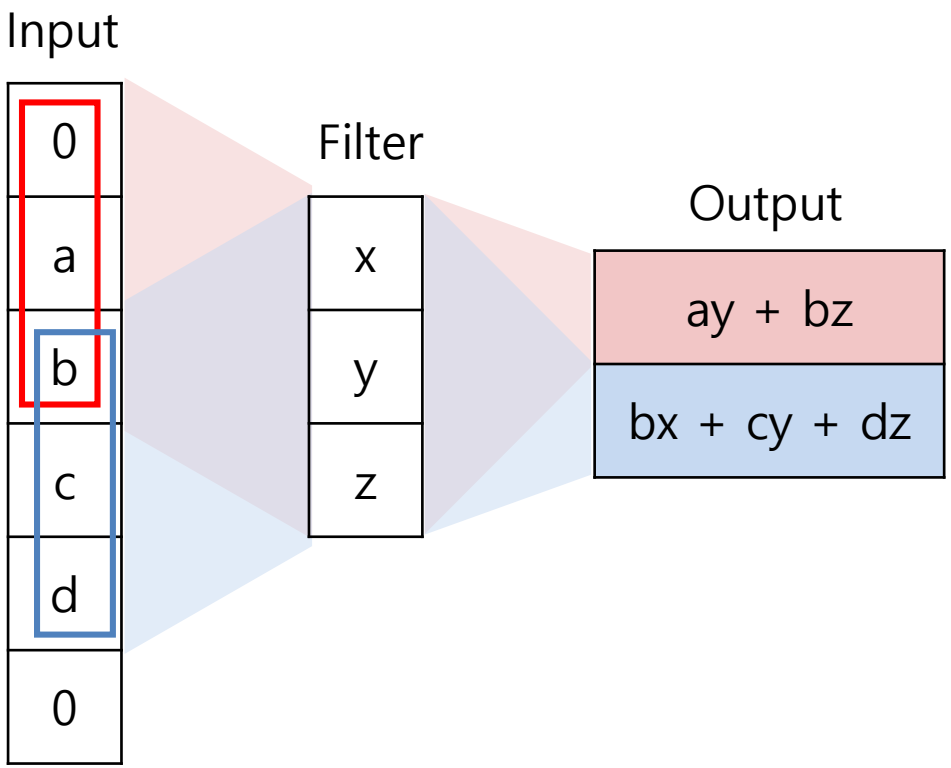
- Key idea: Fully convolutional networks with ***transposed convolution*** (a.k.a. deconvolution)
  - Trainable upsampling with ***transposed convolution*** (a.k.a. deconvolution)
    - e.g. 1D conv (kernel size: 3, stride: 2, padding: 1) vs. 1D transposed conv (kernel size: 3, stride: 2, padding: 0)



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

FCN (Fully Convolutional Networks; 2015)

- Key idea: Fully convolutional networks with ***transposed convolution*** (a.k.a. deconvolution)
  - Trainable upsampling with ***transposed convolution*** (a.k.a. deconvolution)
    - e.g. 1D conv (kernel size: 3, stride: 2, padding: 1) vs. 1D transposed conv (kernel size: 3, stride: 2, padding: 0)



1) Convolution as a matrix multiplication

$$\mathbf{x} * \mathbf{a} = \mathrm{X}\,\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Slide: [Stanford CS231n: Deep Learning for Computer Vision]

FCN (Fully Convolutional Networks; 2015)

- Key idea: Fully convolutional networks with ***transposed convolution*** (a.k.a. deconvolution)
  - Trainable upsampling with ***transposed convolution*** (a.k.a. deconvolution)
    - e.g. 1D conv (kernel size: 3, stride: 2, padding: 1) vs. 1D transposed conv (kernel size: 3, stride: 2, padding: 0)

1) Convolution as a matrix multiplication

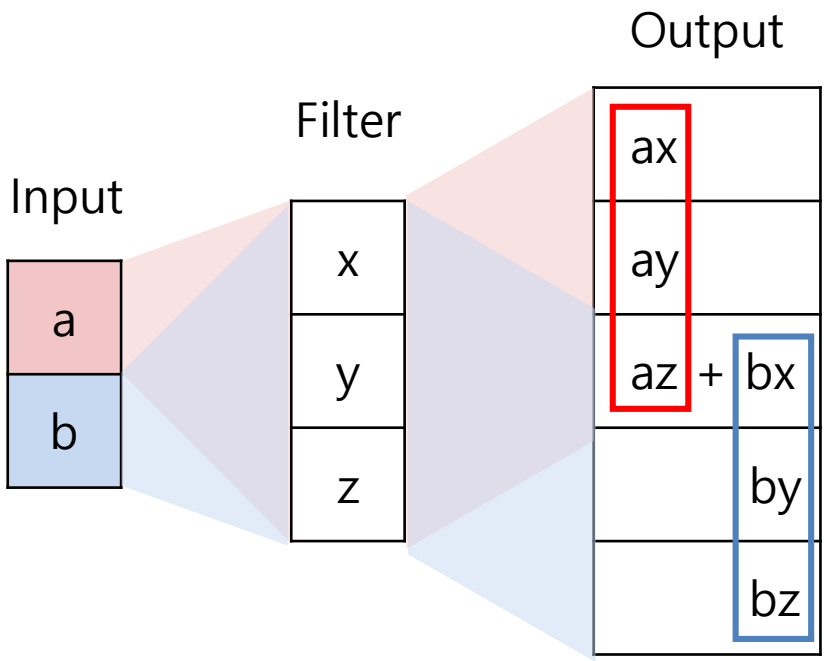
$$\mathbf{x} * \mathbf{a} = \mathbf{X}\,\mathbf{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

2) Transposed convolution

$$\mathbf{x} *^{\mathrm{T}} \mathbf{a} = \mathbf{X}^{\mathrm{T}}\,\mathbf{a}$$

$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$



Slide: [Stanford CS231n: Deep Learning for Computer Vision]

DeconvNet (2015)

- Key idea: A *deep* conv-deconv network with progressive unpooling and deconvolution
  - CNN backbone network: VGG-16 without a classifier (3 x FC layers)

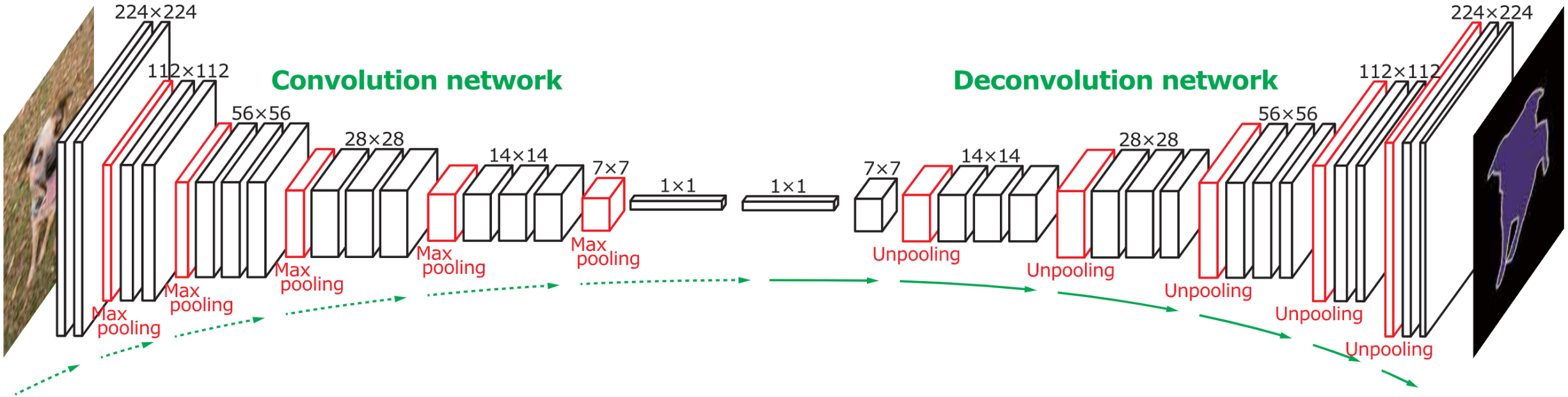
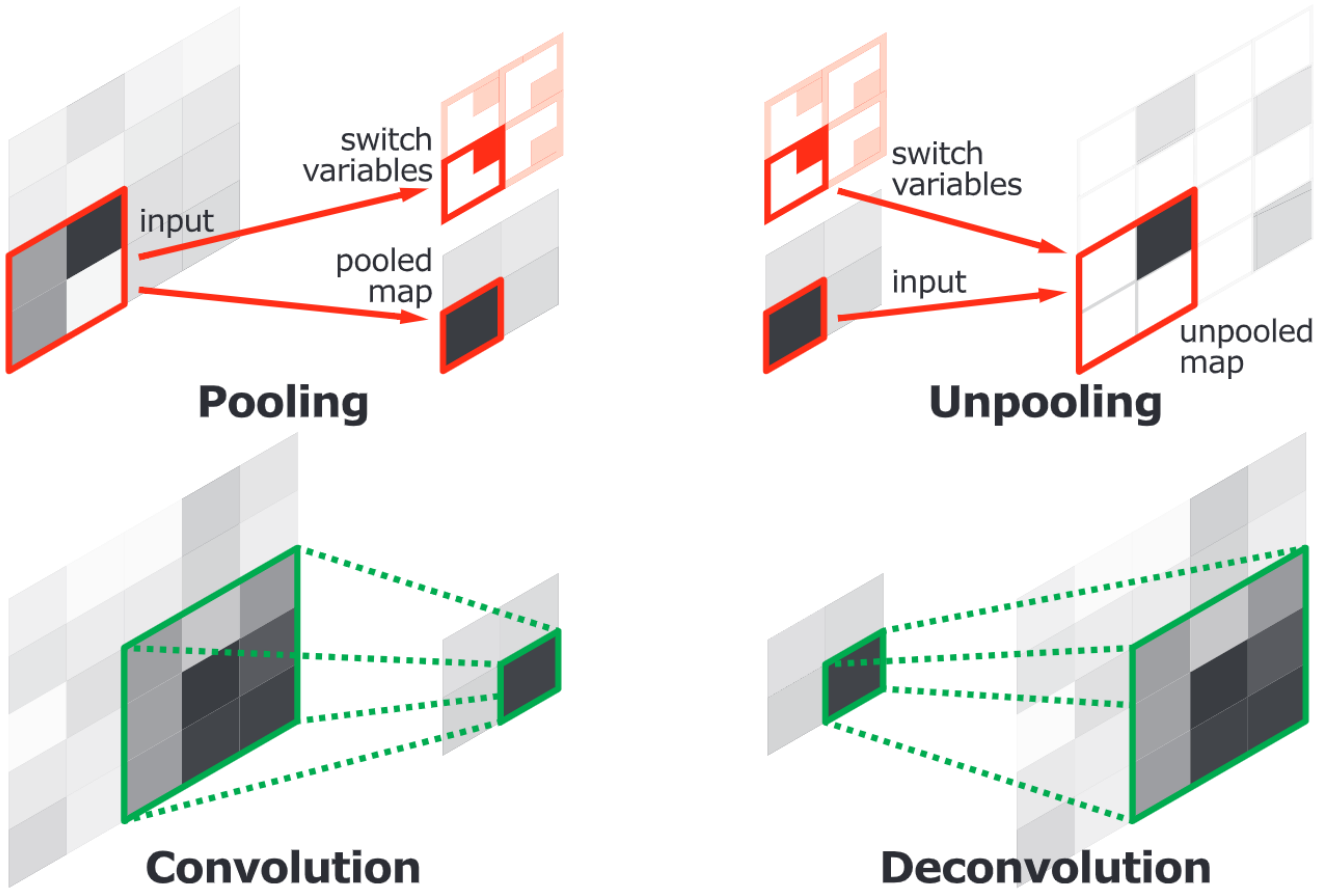


Figure 2. Overall architecture of the proposed network. On top of the convolution network based on VGG 16-layer net, we put a multi-layer deconvolution network to generate the accurate segmentation map of an input proposal. Given a feature representation obtained from the convolution network, dense pixel-wise class prediction map is constructed through multiple series of unpooling, deconvolution and rectification operations.

Image: Noh et al., **Learning Deconvolution Network for Semantic Segmentation**, ICCV, 2015 CVF

DeconvNet (2015)

- Key idea: A *deep* conv-deconv network with progressive unpooling and deconvolution



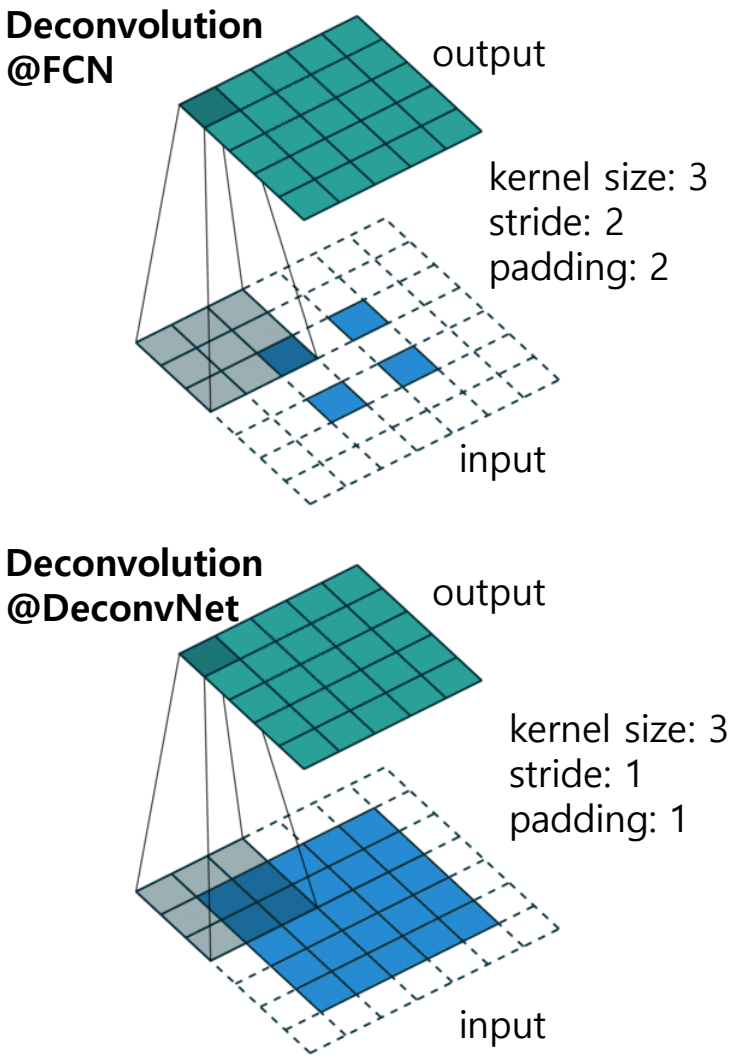
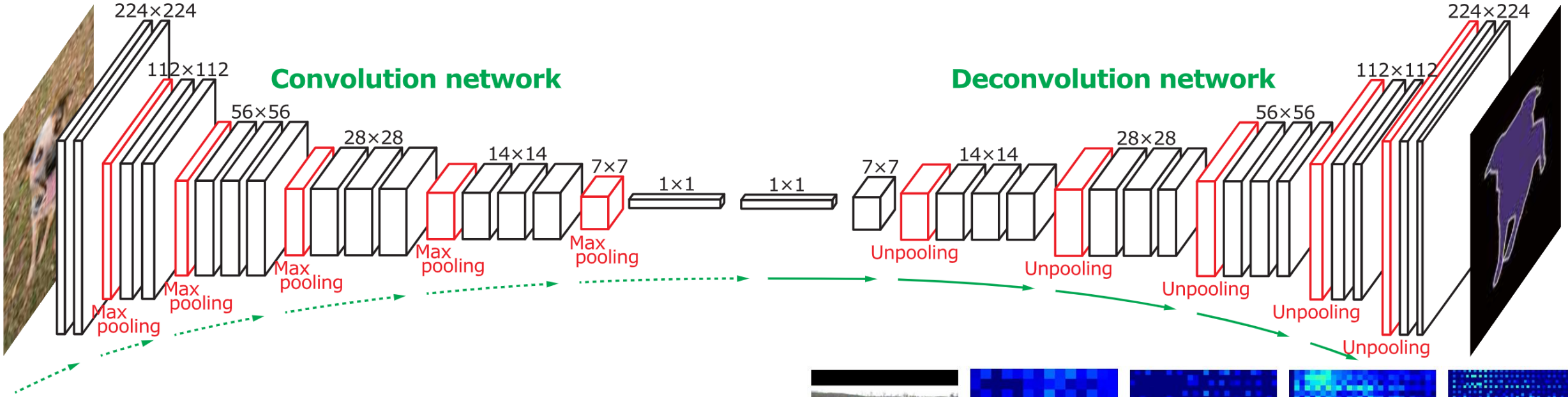


Image: Noh et al., **Learning Deconvolution Network for Semantic Segmentation**, ICCV, 2015 CVF

Image: Convolution arithmetic

DeconvNet (2015)

- Key idea: A *deep* conv-deconv network with progressive unpooling and deconvolution



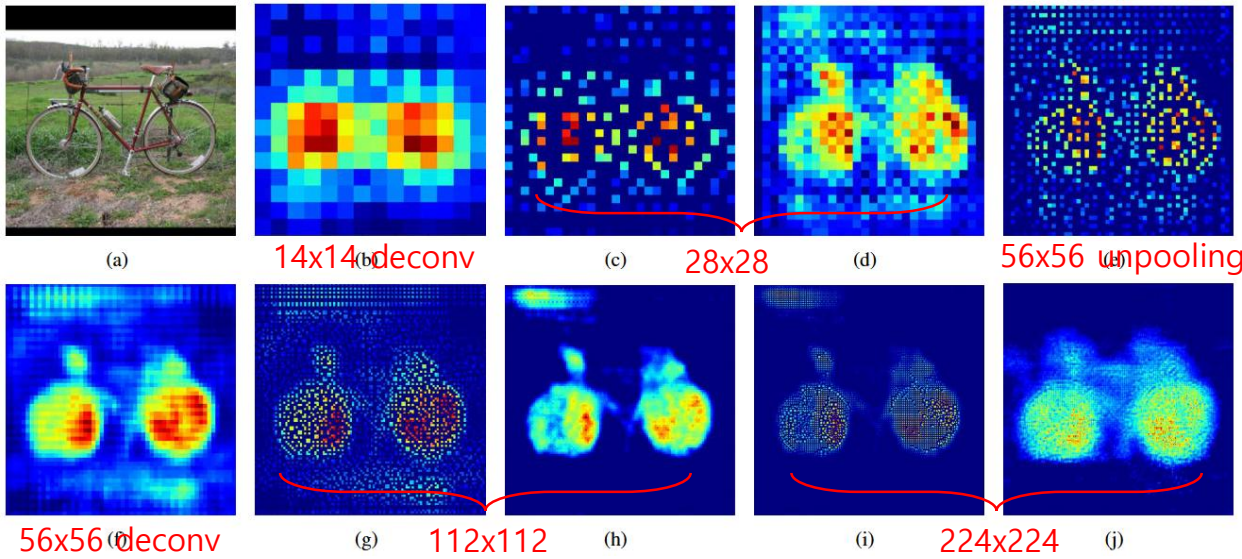


Image: Noh et al., **Learning Deconvolution Network for Semantic Segmen**

U-Net (2015)

- Key idea: A *deep* conv-deconv network that reuses of feature maps

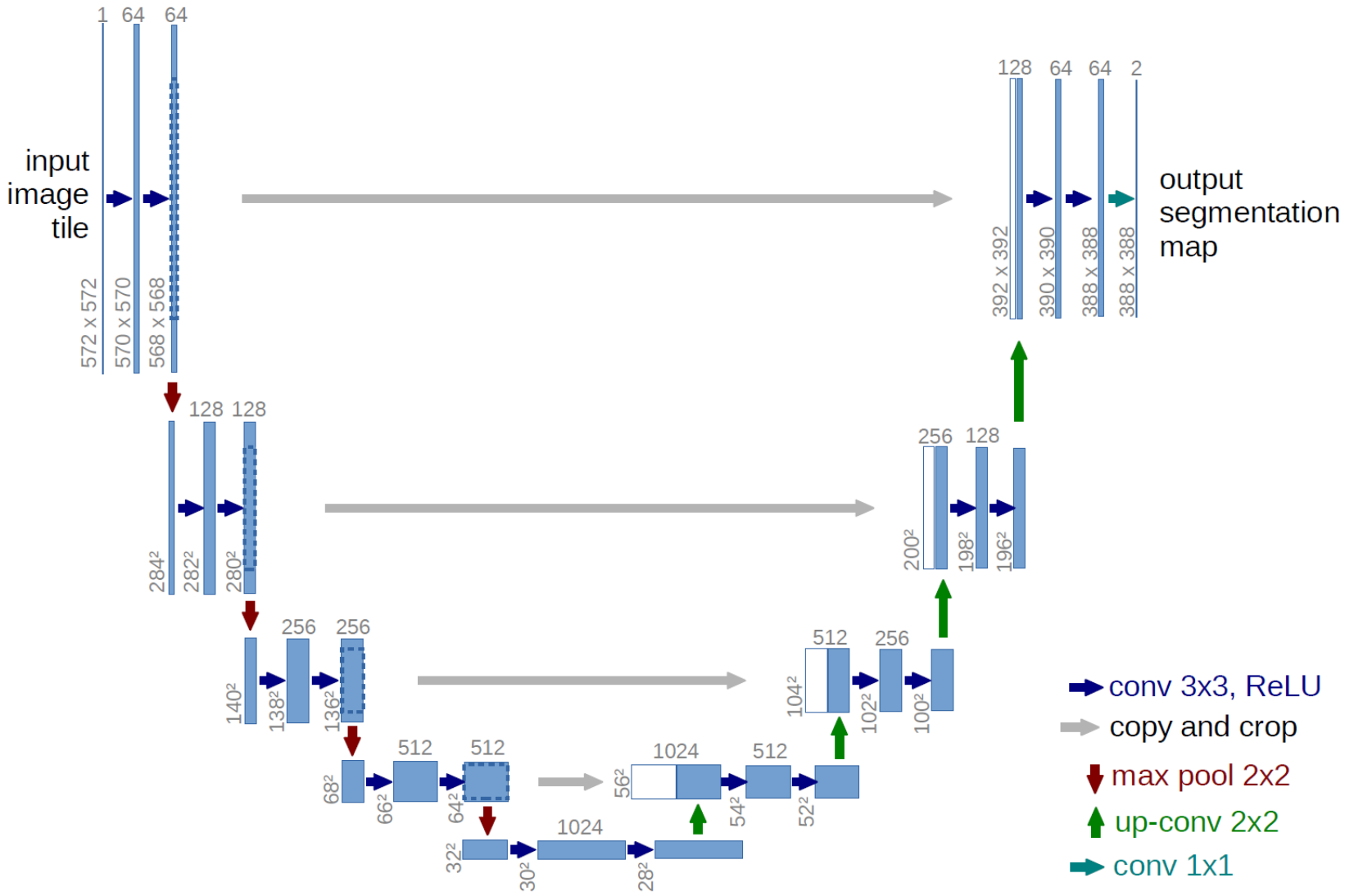
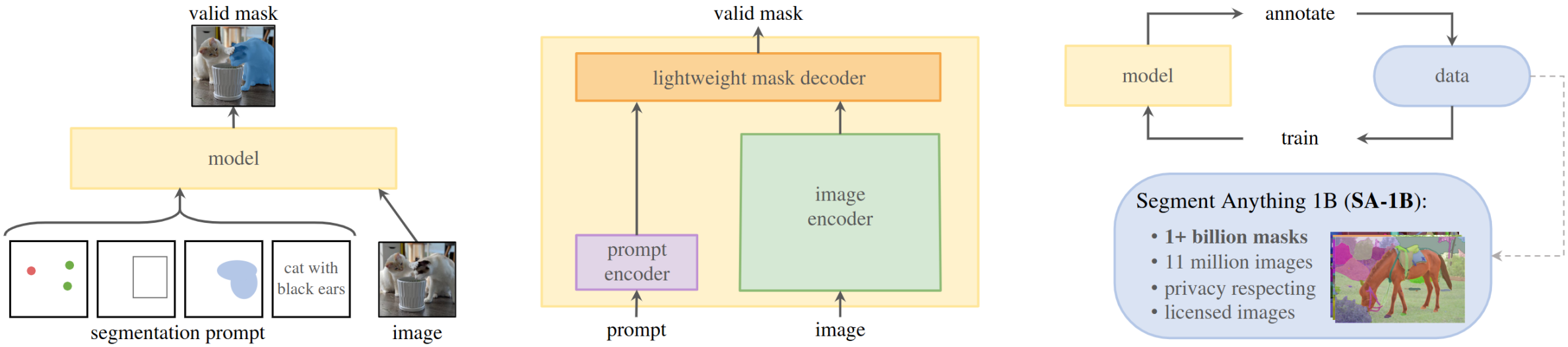


Image: Ronneberger et al., **U-Net: Convolutional Networks for Biomedical Image Segmentation**, MICCAI, 2015 arXiv

SAM (Segment Anything Model; 2023)

- Awesome: A *vision foundation model* for segmentation trained with the SA-1B dataset
  - Image encoder (backbone): ViT (Vision Transformer)
  - Prompts: Point(s), Box, Mask, Text (or automatically generating masks)
  - SA-1B: 1.1B masks on 11M licensed images (Note: The MS COCO dataset contains 0.123M images with 0.9M masks)



(a) **Task**: promptable segmentation (b) **Model**: Segment Anything Model (**SAM**) (c) **Data**: data engine (top) & dataset (bottom)

Figure 1: We aim to build a foundation model for segmentation by introducing three interconnected components: a promptable segmentation *task*, a segmentation *model* (SAM) that powers data annotation and enables zero-shot transfer to a range of tasks via prompt engineering, and a *data* engine for collecting SA-1B, our dataset of over 1 billion masks.

Image: Kirillov et al., **Segment Anything**, arXiv, 2023 Homepage Github

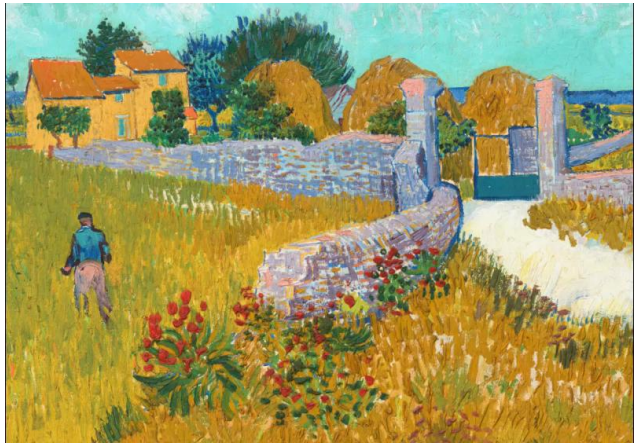
SAM (Segment Anything Model; 2023)

- Awesome: A *vision foundation model* for segmentation trained with the SA-1B dataset
  - Prompts: Point(s), Box, Mask, Text (or automatically generating masks)

Prompt: Point

Prompt: Text

Automatically generated masks



Zero-shot transfer

Image: Kirillov et al., **Segment Anything**, arXiv, 2023 Homepage Github

Summary

- Image segmentation: $\text{labeled_image} = f(\text{image})$

- Fully CNN architectures: Convolution layers + Deconvolution layers (with unpooling)
  - Note) Transfer learning with pretrained CNN models
  - FCN (Fully Convolutional Networks; 2015)
  - DeconvNet (2015)
  - U-Net (2015)

- Note) SAM (Segment Anything Model; 2023)
  - Image encoding using ViT (Vision Transformer)